AI Engineering and Claude Architect · Part IV, Security

AI Security, Exercises

Six Worked Exercises

Threat model an agent, contain an injection, fix a real bug, follow data, do the guardrail arithmetic, and defend a decision.

Module	AI09 of 13
Covers	All 5 lessons
Time	~3 hours

Exercise 1: Threat model an inbox agent

A recruiter's assistant reads a shared inbox (applicants' emails, with CVs attached), reads the candidate database, drafts replies and sends them, and can fetch links from CVs.

(a) Run the trifecta check. **(b)** Name three attacker types and what each can reach. **(c)** Give one tool removal that breaks data theft, and say what it costs. **(d)** Write two abuse-case tests.

Solution

(a) All three legs are present. Private data: candidate database, inbox. Untrusted content: emails and CVs, written by strangers. Ways out: send email, and fetch links, since a URL can carry data. It's high risk, and the state-changing send also forms the second dangerous pair.

(b) An anonymous applicant reaches the inbox, and through it the model. A hiring manager reaches the chat box, and might try to see candidates for other roles. A malicious website, linked from a CV, reaches the model through the fetch tool.

(c) Remove `fetch_url`, or restrict it to a short allowlist, and make `send` draft-only so a person sends each one. The cost: the assistant can't read portfolio links, and sending takes a human step. Draft-only is the stronger of the two, since a send to an arbitrary address is the most direct way out.

(d) One: a scripted model that has read a poisoned CV tries to send the candidate table to an outside address. Assert that no send occurs, and that the attempt is recorded. Two: a scripted model asks for a candidate on another team's role, while the session belongs to a manager without access. Assert the tool refuses. Both test the code, and neither needs a real model.

Exercise 2: Contain the injection

Your agent answers questions about vendor contracts. It can call `search_contracts`, `get_contract`, and `post_to_slack(channel, text)`. Contracts are uploaded by vendors. Security says: "Add a classifier for prompt injection."

(a) What does the classifier give you, and what doesn't it? **(b)** Find the way out, including the non-obvious version. **(c)** Propose changes that keep the useful behaviour. **(d)** What's left over?

Solution

(a) It lowers how often an attack reaches the model. It's a rate, with false alarms that matter for real contracts, which are full of imperative language such as "shall" and "must." It says nothing about what happens when one gets through.

(b) `post_to_slack` is the way out. The obvious version is posting into a channel an attacker reads. The less obvious is that a Slack message containing a link or image may be unfurled, which fetches a URL with no click. A URL can carry data. I haven't tested Slack's behaviour for this exercise, so verify it.

(c) Fix the channel in code, from the session, so the model chooses no destination. Strip links and images from posted text, or disable unfurling. Scope the search to contracts this user can read. Keep untrusted text in `tool_result` blocks, JSON-encoded, and state the policy in the system prompt. Keep the classifier as a rate reducer, sending flags to review rather than blocking uploads.

(d) A misleading summary posted to the right channel, such as "this contract has no termination fee." No destination rule catches that. Answers need citations with a fact check that the quote exists, and consequential decisions need someone to read the clause.

Exercise 3: Fix the ledger

`SecurityLabTest` documents that `ToolLedger.previousResult(toolUseId)` is keyed by ID alone, so a repeated ID in another case returns the first case's stored result.

(a) Change the behaviour so another case can't read it. **(b)** Decide what `begin` should do on such a collision. **(c)** Say how severe this is.

Solution

(a) Scope the read by case:

```
SELECT result FROM tool_calls WHERE tool_use_id = ? AND case_id = ?
```

`previousResult` then takes both values, and `AgentService` passes `context.caseId()`.

(b) The primary key is `tool_use_id`, so `begin` returns false for a collision, and the call doesn't run. That's the safe direction for side effects, and with (a) the second case now gets "Already in progress" instead of another's data. Better still, make it visible: if the row's `case_id` differs from the caller's, log it as a security event and return an error. A cross-case ID shouldn't happen, so when one does, it needs investigating. Update the test to assert the new behaviour.

(c) Low as far as I can tell, since IDs are generated by the API rather than chosen in model text, which I haven't tested against the real API. Fixing it still matters, because stored results can contain personal data, and the fix costs one column in a query. It's also the same pattern as the semantic cache in L4: anything replayed needs the same scope as its original request.

Exercise 4: Follow the data

A healthcare scheduling assistant, multi-tenant across clinics, has: a Postgres database, a vector index of clinic policies, a semantic cache, debug logging of full prompts, a nightly Batch evaluation of production conversations, and agent memory per patient.

(a) Identify three risks. **(b)** The organization has ZDR. What's the concern with this list? **(c)** A patient asks for deletion: list what must be done.

Solution

(a) Any three of these. The semantic cache's key may omit the clinic, so one clinic's answer reaches another. Debug logs hold health information, in a store with wide staff access. Evaluation data is production conversations copied elsewhere with no retention rule. Memory is an injection path as well as a store. And the vector index needs a clinic filter, taken from the session.

(b) Per the data retention page, Batch isn't ZDR-eligible, and has 29-day retention. Under ZDR the API doesn't block it, so the nightly job quietly takes this data outside the arrangement. If the organization relies on HIPAA readiness rather than ZDR, the page says that ineligible features return a 400. Either way, take this to compliance. Use synthetic or de-identified evaluation data, or run the evaluation without Batch. This isn't legal advice.

(c) Database rows, agent memory, cache entries derived from them (scoped by user or expired), logs and traces (searchable by user, or by retention expiry), evaluation sets, provider-held files or Batch results if any, and backups by their window. Then record what was done. The policies in the vector index aren't patient data, so they stay.

Exercise 5: Guardrail arithmetic

Your product gets 2,000,000 messages a month. Red-team analysis suggests about 0.2% are attacks (assume this for the exercise). Vendor A: recall 98%, false alarms 3%. Vendor B: recall 90%, false alarms 0.5%.

(a) Compute missed attacks, innocent flags, and the share of flags that are real, for each. **(b)** If a flag means a hard block, which would you choose? **(c)** If a flag means sending to stricter handling at $0.01 each, how much does each cost monthly? **(d)** If both screens are the same type, should you run both?

Solution

(a) Attacks: 4,000. Innocent: 1,996,000.

	Caught	Missed	Innocent flagged	Real share
A	3,920	80	59,880	6.1%
B	3,600	400	9,980	26.5%

(b) B. A blocks 59,880 innocent messages each month. B misses more attacks, and those have to be contained by tools and policy anyway.

(c) A: 63,800 flags × $0.01 = $638. B: 13,580 × $0.01 = $135.80. With stricter handling instead of a block, false alarms cost little, so A becomes reasonable: for about $500 more, it catches 320 more attacks. This shows how the decision depends on what a flag does.

(d) Not on the arithmetic alone. The independent figure would be 0.02 × 0.10 = 0.2% missed, but two similar classifiers tend to fail together, so measure on your own red-team set. A rule or fact check beside one classifier is likely to add more.

Exercise 6: Defend a decision

The decision: a fintech's agent can issue refunds up to $200 with no human approval, after reading customer tickets. A security reviewer wants every refund to need approval, and the head of support says that will destroy the value of the agent. Write the case you'd make for a middle position, with its risks.

Model answer

I'd keep autonomous refunds, inside limits that hold when the model is fooled, and I'd accept a defined loss.

The threat. Tickets are untrusted content, and `process_refund` changes state, so this is the second dangerous pair. Assume injection will sometimes succeed. Anthropic doesn't claim immunity for its models, and I haven't measured a rate for ours. So the question is what success can cost.

What code guarantees. The refund goes only to the original payment method, so an attacker can't redirect it. Only for this customer's order, with identity from the session. Never above the amount paid, in total across calls, enforced by a constraint in the payment service. Per-customer and per-day limits. L3's lab showed that the first version of such a policy had four defects, so a scripted compromised model runs in CI.

Why this answers the reviewer. The worst case for one attack is a refund of money to the customer's own card, on their own order. The attacker gains little unless they're the customer, and a customer attacking their own ticket is ordinary refund fraud, which the business already handles with limits and monitoring.

Why this answers support. Most refunds run at once. Approval remains for what's unusual: above the limit, a closed account, a burst of refunds, or a ticket flagged by the screen. That keeps prompts rare enough to be read, which is the argument Anthropic makes on approval fatigue.

Risks I'm accepting. Refund fraud within limits, which should be sized from real data, not from my guess. A false sense of safety if the backend rule drifts from the policy. And a misleading message to a customer, which no limit prevents.

What would change my mind. Fraud above the accepted budget. A way to redirect funds. A rise in refunds not explained by volume. Or evaluation by segment showing that some customers get worse outcomes. Then I'd lower the limit, or require approval for that group, and I'd start with a low limit and raise it on evidence.